

Documentation Cybersécurité

Objectif du projet

Ce projet cherche à concevoir et développer un système permettant d'assurer la sauvegarde automatisée et la restauration automatisées d'une base de données MariaDB. Ce projet est à réaliser en équipe en utilisant la méthode SCRUM

Les choix techniques

Pour le système d'exploitation, nous avons utilisé un iso Ubuntu car nous l'avons déjà installé et utilisé avoir la gestion d'une base de données MariaDB.

Nous avons décidé d'utiliser des machines virtuelles sur VMware Workstation pour les mêmes raisons et aussi car c'est un environnement isolé et il est facilement reproduit.

Pour la base de données, nous avons utilisé MariaDB comme indiqué dans le sujet.

Nous avons utilisé le langage bash pour rendre plus facile l'automatisation des sauvegardes et aussi car nous utilisons Ubuntu, donc Linux, et que bash est natif sur Linux.

Pour le chiffrement, nous avons utilisé le GPG RSA 4096 bits pour sa sécurité asymétrique et donc que le fichier reste illisible et crypté sans la clé privée.

Pour le transfert des sauvegardes entre les machines, nous avons utilisé SSH et Rsync. Le SSH permet de créer une connexion privée entre les deux machines virtuelles. Et Rsync permet de copier les fichiers et transfert seulement les blocs modifiés.

Pour l'automatisation, nous avons utilisé cron qui est déjà intégré à Linux, et qui permet d'exécuter le système automatiquement à intervalles réguliers.

Pour vérifier l'intégrité des fichiers sauvegardés, nous avons utilisé SHA-256 pour vérifier que les fichiers soient toujours complets après la sauvegarde.

Configuration de l'environnement

Nous avons utilisé deux machines virtuelles pour ce projet, une machine srv-db pour héberger la base de données. Son adresse IP est 192.168.75.129.

Puis une autre machine srv-backup pour recevoir les sauvegardes. Son adresse IP est : 192.168.75.133.

Les étapes d'installation

Etape 1 : mise à jour des paquets sur les deux machines

sudo apt update && sudo apt upgrade -y

```
clement@srv-db:~$ sudo apt update
sudo apt upgrade -y
sudo apt install -y curl wget vim git unzip
[sudo] Mot de passe de clement :
```

Etape 2 : installation des outils nécessaires

`sudo apt install -y mariadb-server mariadb-client gnupg rsync openssh-client` (sur srv-db)

Pour installer et utiliser MariaDB, GPG, Rsync et SSH

`sudo apt install -y openssh-server rsync`

`sudo systemctl enable --now ssh`

Sur srv-backup pour installer SSH, Rsync et activer le SSH

`sudo systemctl status mariadb`

`sudo systemctl status ssh`

Pour vérifier que tout est bien installer

```
clement@srv-db:~$ sudo systemctl status mariadb
● mariadb.service - MariaDB 10.11.13 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: >
   Active: active (running) since Fri 2025-11-07 19:47:52 CET; 28s ago
     Docs: man:mariadb(8)
           https://mariadb.com/kb/en/library/systemd/
   Process: 5028 ExecStartPre=/usr/bin/install -m 755 -o mysql -g root -d /var>
   Process: 5030 ExecStartPre=/bin/sh -c systemctl unset-environment _WSREP_ST>
   Process: 5033 ExecStartPre=/bin/sh -c [ ! -e /usr/bin/galera_recovery ] &&>
   Process: 5105 ExecStartPost=/bin/sh -c systemctl unset-environment _WSREP_S>
   Process: 5107 ExecStartPost=/etc/mysql/debian-start (code=exited, status=0/>
  Main PID: 5093 (mariabdb)
    Status: "Taking your SQL requests now..."
     Tasks: 13 (limit: 29986)
    Memory: 78.8M (peak: 82.1M)
       CPU: 297ms
    CGroup: /system.slice/mariadb.service
            └─5093 /usr/sbin/mariabdb

nov. 07 19:47:52 srv-db mariabdb[5093]: 2025-11-07 19:47:52 0 [Note] InnoDB: lo>
nov. 07 19:47:52 srv-db mariabdb[5093]: 2025-11-07 19:47:52 0 [Note] InnoDB: Lo>
nov. 07 19:47:52 srv-db mariabdb[5093]: 2025-11-07 19:47:52 0 [Note] Plugin 'FE>
```

Etape 3 : configuration de MariaDB

`sudo mysql_secure_installation`

Pour augmenter la sécurité de la base de données en demandant par exemple de définir un mot de passe root, ou encore supprimer les utilisateurs anonymes.

```
CREATE DATABASE projet_backup;
```

```
CREATE USER 'backup'@'localhost' IDENTIFIED BY 'password';
```

```
GRANT ALL PRIVILEGES ON projet_backup.* TO 'backup'@'localhost';
```

```
FLUSH PRIVILEGES;
```

```
EXIT;
```

Pour créer la base de données, ici appelé projet_backup, et aussi l'utilisateur appelé backup et lui donner tous les droits.

```

clement@srv-db:~$ sudo mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 41
Server version: 10.11.13-MariaDB-0ubuntu0.24.04.1 Ubuntu 24.04

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE projet_backup;
Query OK, 1 row affected (0,002 sec)

MariaDB [(none)]> CREATE USER 'backup'@'localhost' IDENTIFIED BY 'Backup123!';
Query OK, 0 rows affected (0,005 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON projet_backup.* TO 'backup'@'localhost';
Query OK, 0 rows affected (0,002 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0,001 sec)

```

Etape 4 : génération de la clé GPG

gpg --full-generate-key

Type : RSA and RSA

Taille : 4096 bits

Expiration : aucune

Nom : backup

Email : backup@exemple.org

Passphrase : vide (pour l'automatiser)

La commande pour vérifier : gpg --list-keys

```

clement@srv-db:~$ gpg --list-keys
gpg: vérification de la base de confiance
gpg: marginals needed: 3 completes needed: 1 trust model: pgp
gpg: profondeur : 0 valables : 1 signées : 0
confiance : 0 i., 0 n.d., 0 j., 0 m., 0 t., 1 u.
/home/clement/.gnupg/pubring.kbx
-----
pub   rsa4096 2025-11-07 [SC]
      40A22A02E1314A40F86E3CD8F40489DCB5505714
uid           [  ultime ] backup <backup@exemple.org>
sub   rsa4096 2025-11-07 [E]

```

Etape 5 : configuration SSH sans mot de passe

Sur le srv-db :

```
ssh-keygen -t ed25519
```

```
ssh-copy-id -i ~/.ssh/id_ed25519 clement@192.168.75.133
```

```
ssh -i ~/.ssh/id_ed25519 clement@192.168.75.133
```

Pour générer une clé publique et une clé privée, on copie ensuite la clé publique sur l'autre machine alors que la clé privée reste sur le srv-db. Et on teste la connexion SSH, qui fonctionne si elle se fait sans nous demander de mot de passe.

```
Your identification has been saved in /home/clement/.ssh/id_ed25519
Your public key has been saved in /home/clement/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:n60HpaeOY976/NkGs/6IGilivLTG0KZtTBTeQQL/eME clement@srv-db
The key's randomart image is:
+--[ED25519 256]--+
|  . . . . .      |
|  ..o.           |
|  ..oE.          |
|   oo..   .      |
|  o. o S  o      |
| ..+. . 000+     |
| 0= . o o+.+    |
| .o*+  o*.00+.   |
|  oo  o==B+=oo   |
+-----[SHA256]-----+
```

Etape 6 : création du projet de sauvegarde

Sur le srv-db

```
mkdir -p ~/projet_sauvegarde/{config,scripts,logs}
```

```
sudo mkdir -p /var/backups/mariadb /var/log/backup-mariadb
```

```
sudo chown $USER:$USER /var/backups/mariadb /var/log/backup-mariadb
```

Pour créer l'arborescence du projet, les dossiers des sauvegardes et des logs, et attribuent les bons droits à l'utilisateur pour qu'il puisse exécuter les scripts sans avoir de problème de permission.

```
GNU nano 7.2 /home/clement/projet_sauvegarde/config/.env *
DB_NAME="projet_backup"
DB_USER="backup"
DB_HOST="127.0.0.1"
DB_PORT=3306

BACKUP_DIR="/var/backups/mariadb"
GPG_RECIPIENT="backup@exemple.org"

REMOTE_USER="clement"
REMOTE_HOST="192.168.75.133"
REMOTE_DIR="/data/backups/mariadb"

SSH_KEY="/home/clement/.ssh/id_ed25519"
RETENTION_DAYS=14
LOG_DIR="/var/log/backup-mariadb"
JOB_NAME="mariadb-prod"

```

^G Aide	^O Écrire	^W Chercher	^K Couper	^T Exécuter	^C Emplacement
^X Quitter	^R Lire fich.	^\ Remplacer	^U Coller	^J Justifier	^/ Aller ligne

Le fichier .env contient toutes les configurations nécessaires au script, comme les informations de connexion à la base, les chemins de sauvegarde et de logs, les paramètres du chiffrement GPG, et les données du serveur distant pour le transfert sécurisé.

Etape 7 : le script de sauvegarde
Script appelé backup.sh

```
GNU nano 7.2 /home/clement/projet_sauvegarde/scripts/backup.sh *
#!/bin/bash
set -e

# Charger variables
source ~/projet_sauvegarde/config/.env

DATE=$(date +%Y-%m-%d_%H-%M-%S)
BACKUP_FILE="${BACKUP_DIR}/${DB_NAME}_${DATE}.sql.gz"
ENCRYPTED_FILE="${BACKUP_FILE}.pgp"

echo "[INFO] Dump de la base ${DB_NAME}..."
mysqldump -u${DB_USER} ${DB_NAME} | gzip > "${BACKUP_FILE}"

echo "[INFO] Chiffrement du dump..."
gpg --yes --encrypt --recipient "${GPG_RECIPIENT}" "${BACKUP_FILE}"

echo "[INFO] Transfert du fichier vers le serveur distant..."
rsync -avz -e "ssh -i ${SSH_KEY}" "${ENCRYPTED_FILE}" ${REMOTE_USER}@${REMOTE_HOST}:${REMOTE_DIR}/

echo "[INFO] Vérification d'intégrité..."
LOCAL_HASH=$(sha256sum "${ENCRYPTED_FILE}" | awk '{print $1}')
REMOTE_HASH=$(ssh -i ${SSH_KEY} ${REMOTE_USER}@${REMOTE_HOST} "sha256sum ${REMOTE_DIR}/${(basename ${ENCRYPTED_FILE})}")

if [ "$LOCAL_HASH" == "$REMOTE_HASH" ]; then
    echo "[OK] Vérification d'intégrité réussie."
else
    echo "[ERREUR] Vérification échouée !" >&2
fi

find "${BACKUP_DIR}" -type f -mtime +${RETENTION_DAYS} -delete

^G Aide      ^O Écrire    ^W Chercher  ^K Couper    ^T Exécuter  ^C Emplacement M-U Annuler
^X Quitter   ^R Lire fich.^_ Remplacer  ^U Coller    ^J Justifier ^_ Aller ligne M-E Refaire
```

chmod +x ~/projet_sauvegarde/scripts/backup.sh
La commande permet de rendre le script exécutable.

```
clement@srv-db:~$ ls -l ~/projet_sauvegarde/scripts/
total 4
-rwxrwxr-x 1 clement clement 1118 nov.  7 22:59 backup.sh
```

Etape 8 : préparation du serveur distant

Sur srv-backup

sudo mkdir -p /data/backups/mariadb

sudo chown \$USER:\$USER /data/backups/mariadb

rsync -avz -e "ssh -i ~/.ssh/id_ed25519" /etc/hostname

clement@192.168.75.133:/data/backups/mariadb/

Ces commandes créent le dossier de sauvegarde sur le serveur distant, lui attribuent les bons droits d'accès, et testent le transfert avec rsync pour vérifier que les fichiers peuvent être copiés correctement avec une connexion SSH sécurisée.

Etape 9 : test de sauvegarde complète

cd ~/projet_sauvegarde/scripts

./backup.sh

Ces commandes permettent d'accéder au dossier des scripts et d'exécuter le script backup.sh, qui lance la sauvegarde, le chiffrement et le transfert vers le srv-backup.

```

clement@srv-db:~/projet_sauvegarde/scripts$ ./backup.sh
[INFO] Dump de la base projet_backup...
[INFO] Chiffrement du dump...
[INFO] Transfert du fichier vers le serveur distant...
Enter passphrase for key '/home/clement/.ssh/id_ed25519':
sending incremental file list
projet_backup_2025-11-07_23-48-32.sql.gz.gpg

sent 1.284 bytes  received 35 bytes  175,87 bytes/sec
total size is 1.131  speedup is 0,86
[INFO] Vérification d'intégrité...
Enter passphrase for key '/home/clement/.ssh/id_ed25519':
[OK] Vérification d'intégrité réussie.

```

On peut voir sur le serveur distant

```

clement@srv-db:~$ ls -lh /data/backups/mariadb
total 20K
-rw-r--r-- 1 clement clement  7 oct.  29 16:15 hostname
-rw-rw-r-- 1 clement clement 648 nov.  7 23:30 projet_backup_2025-11-07_23-30-55.s
ql.gz.gpg
-rw-rw-r-- 1 clement clement 648 nov.  7 23:41 projet_backup_2025-11-07_23-41-01.s
ql.gz.gpg
-rw-rw-r-- 1 clement clement 648 nov.  7 23:45 projet_backup_2025-11-07_23-45-10.s
ql.gz.gpg
-rw-rw-r-- 1 clement clement 1,2K nov.  7 23:48 projet_backup_2025-11-07_23-48-32.s
ql.gz.gpg

```

Etape 10 : script de restauration

Script appelé restore.sh

```

GNU nano 7.2 /home/clement/projet_sauvegarde/scripts/restore.sh *
#!/bin/bash
set -e
source ~/projet_sauvegarde/config/.env

LATEST=$(ssh -i ${SSH_KEY} ${REMOTE_USER}@${REMOTE_HOST} "ls -t ${REMOTE_DIR}/*.gpg | head -1")
scp -i ${SSH_KEY} ${REMOTE_USER}@${REMOTE_HOST}:${LATEST} /tmp/

gpg --output /tmp/restore.sql.gz --decrypt /tmp/${(basename ${LATEST})}
gunzip /tmp/restore.sql.gz
mysql -u${DB_USER} ${DB_NAME} < /tmp/restore.sql

echo "[OK] Restauration terminée."

```

chmod +x ~/projet_sauvegarde/scripts/restore.sh

Comme au-dessus, la commande sert à rendre le script exécutable.

Pour vérifier que la restauration fonctionne

```
clement@srv-db:~/projet_sauvegarde/scripts$ ./restore.sh
Enter passphrase for key '/home/clement/.ssh/id_ed25519':
Enter passphrase for key '/home/clement/.ssh/id_ed25519':
projet_backup_2025-11-08_00-19-07.sql.gz.gpg                                100% 1131      1.4MB/s   00:00
gpg: encrypted with rsa4096 key, ID 1D5D054D1840AE95, created 2025-11-07
      « backup <backup@exemple.org> »
gzip: /tmp/restore.sql already exists; do you wish to overwrite (y or n)? y
[OK] Restauration terminée.
```

Etape 11 : automatiser avec cron

`crontab -e 0 2 * * * /home/clement/projet_sauvegarde/scripts/backup.sh`

La commande permet d'ouvrir le planificateur cron et d'y ajouter le script backup.sh pour le rendre automatique chaque jour à 2 heures du matin.

```
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

# Sauvegarde MariaDB automatique chaque jour à 2h00
0 2 * * * /home/clement/projet_sauvegarde/scripts/backup.sh >> /var/log/backup-mariadb/backup.log 2>&1
```

Etape 12 : vérification des logs

`tail -n 10 /var/log/backup-mariadb/backup.log`

La commande permet d'afficher les 10 dernières lignes du fichier de log pour détecter de possibles erreurs et aussi le déroulement des dernières sauvegardes.